C Programming Activity

Variadic Function and Command-Line Arguments

Name	Gurveer Singh Cheema
SAF ID	590032925
Batch	15

Program 1 - Weighted Average Using a Variadic Function

Aim: To write a C program containing the variadic function **double weightedAverage(int count, ...)** that accepts value-weight pairs, validates the weights, and returns the weighted average.

Algorithm

1. Start the variadic function with the number of value-weight pairs.
2. Initialize **va_list** using **va_start()**.
3. Read each value and its weight using **va_arg()**.
4. If any weight is negative, display an error and return an invalid result.
5. Calculate **sum(value x weight)** and total weight.
6. If total weight is zero, display an error message.
7. Otherwise return weighted sum / total weight.
8. End variable argument processing using **va_end()**.

C Program

```
#include <stdio.h>
#include <stdarg.h>

double weightedAverage(int count, ...)
{
    va_list args;
    double value, weight;
    double weightedSum = 0.0, totalWeight = 0.0;
    int i;

    if (count <= 0)
    {
        printf("Error: count must be positive.\n");
        return -1.0;
    }

    va_start(args, count);

    for (i = 0; i < count; i++)
    {
        value = va_arg(args, double);
        weight = va_arg(args, double);

        if (weight < 0)
        {
            printf("Error: weights cannot be negative.\n");
            va_end(args);
            return -1.0;
        }

        weightedSum += value * weight;
        totalWeight += weight;
    }

    va_end(args);

    if (totalWeight == 0.0)
    {
```

```
        printf("Error: total weight cannot be zero.\n");
        return -1.0;
    }

    return weightedSum / totalWeight;
}

int main()
{
    double result;

    result = weightedAverage(3,
                             80.0, 0.3,
                             75.0, 0.2,
                             90.0, 0.5);

    if (result >= 0)
        printf("Weighted Average = %.2f\n", result);

    return 0;
}
```

Sample Output

```
Weighted Average = 84.00
```

Calculation: (80 x 0.3 + 75 x 0.2 + 90 x 0.5) / (0.3 + 0.2 + 0.5) = 84.00

Explanation

<stdarg.h> is used for functions that can accept a variable number of arguments. **va_start()** starts reading the extra arguments, **va_arg()** retrieves each argument, and **va_end()** finishes processing them. The program also rejects negative weights and prevents division by zero.

Result

Thus, the weighted average was calculated successfully using a variadic function with proper input validation.

Program 2 - Processing Command-Line Integer Arguments

Aim: To write a C program that accepts any number of command-line arguments, ignores invalid non-integer arguments, sorts valid integers, and calculates the minimum, maximum, average, median, and second-largest distinct value.

Algorithm

1. Read command-line arguments through **argc** and **argv**.
2. Validate every argument using a separate function.
3. Ignore and report arguments that are not complete integers.
4. Store all valid integers in an array.
5. Sort the valid integers in ascending order.
6. The first value is the minimum and the last value is the maximum.
7. Calculate average = sum / number of valid integers.
8. Calculate the median according to whether the count is odd or even.
9. Search backward for the first value smaller than the maximum to obtain the second-largest distinct value.

C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <limits.h>

int isValidInteger(const char *str, int *value)
{
    char *end;
    long num;

    if (str == NULL || *str == '\0')
        return 0;

    errno = 0;
    num = strtol(str, &end, 10);

    if (errno == ERANGE || num < INT_MIN || num > INT_MAX)
        return 0;

    if (*end != '\0')
        return 0;

    *value = (int)num;
    return 1;
}

void sortAscending(int a[], int n)
{
    int i, j, temp;

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

double findAverage(int a[], int n)
{
    long long sum = 0;
    int i;

    for (i = 0; i < n; i++)
        sum += a[i];
```

```
    return (double)sum / n;
}

double findMedian(int a[], int n)
{
    if (n % 2 != 0)
        return a[n / 2];

    return ((double)a[n / 2 - 1] + a[n / 2]) / 2.0;
}

int findSecondLargestDistinct(int a[], int n, int *found)
{
    int i;

    for (i = n - 2; i >= 0; i--)
    {
        if (a[i] != a[n - 1])
        {
            *found = 1;
            return a[i];
        }
    }

    *found = 0;
    return 0;
}

int main(int argc, char *argv[])
{
    int *numbers;
    int count = 0, value, i;
    int found, secondLargest;

    if (argc < 2)
    {
        printf("No arguments supplied.\n");
        return 0;
    }

    numbers = (int *)malloc((argc - 1) * sizeof(int));

    if (numbers == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (i = 1; i < argc; i++)
    {
        if (isValidInteger(argv[i], &value))
            numbers[count++] = value;
        else
            printf("Invalid argument ignored: %s\n", argv[i]);
    }

    if (count == 0)
    {
        printf("No valid integers found.\n");
        free(numbers);
        return 0;
    }

    sortAscending(numbers, count);

    printf("Valid integers in ascending order: ");
    for (i = 0; i < count; i++)
        printf("%d ", numbers[i]);
    printf("\n");

    printf("Minimum: %d\n", numbers[0]);
    printf("Maximum: %d\n", numbers[count - 1]);
    printf("Average: %.2f\n", findAverage(numbers, count));
    printf("Median: %.2f\n", findMedian(numbers, count));

    secondLargest = findSecondLargestDistinct(numbers, count, &found);

    if (found)
```

```
        printf("Second-largest distinct value: %d\n", secondLargest);
    else
        printf("Second-largest distinct value does not exist.\n");

    free(numbers);
    return 0;
}
```

Compilation and Input

```
cc program.c -o program
./program 12 5 abc 8 12 20 3
```

Sample Output

```
Invalid argument ignored: abc
Valid integers in ascending order: 3 5 8 12 12 20
Minimum: 3
Maximum: 20
Average: 10.00
Median: 10.00
Second-largest distinct value: 12
```

Explanation

argc stores the number of command-line arguments and **argv** stores them as strings. **strtol()** is used to validate and convert each argument safely. Bubble sort arranges valid integers in ascending order. Separate functions are used for validation, sorting, average, median, and second-largest distinct value.

Result

Thus, command-line arguments were validated and processed successfully, and the required statistical values were displayed.

Quick Viva Questions

1. What is a variadic function? A function that accepts a variable number of arguments.
2. Which header is required for variadic functions? <stdarg.h>.
3. What are argc and argv? argc is the argument count; argv is the array of argument strings.
4. Why is strtol() useful here? It converts a string to a long integer and allows validation of the complete input.
5. What is a distinct value? A value different from another value; duplicates are not counted separately.
6. Why is the median 10.00 in the example? There are six sorted values, so median = (8 + 12) / 2 = 10.00.